

One small note: \$1, \$2, \$3, etc. are placeholders for values supplied by an application. For example, \$1 could later be replaced by a specific genre_id. When we manually test the queries, we can substitute actual values.

And the last two queries are simple demonstration for easier explanation and testing.

Query 1 — Find a specific track within an album

Type: Point query

Description:

This query looks for one specific track when we already know the album and track name. For example, if a user opens an album and clicks on one song, the application could use this query to display information about that track.

The query first finds the track in tracks, then joins to album to obtain the album name, genre to obtain the genre name, and track_feel to obtain musical characteristics such as energy, tempo, and danceability. It also finds all artists associated with the track and combines their names into one field.

Because the combination of an album and a track name should normally identify only one or a very small number of rows, this is a highly selective point query.

SQL:

```
SELECT
  t.track_id,
  t.track_name,
  al.album_name,
  g.genre_name,
  t.popularity,
  t.duration_ms,
  t.explicit,
  tf.danceability,
  tf.energy,  tf.key,
  tf.loudness,
  tf.acousticness,
  tf.instrumentalness,
  tf.liveness,
  tf.valence,
  tf.tempo,
  (
```

```

        SELECT STRING_AGG(ar.artist_name, ', ' ORDER BY ar.artist_name)
FROM track_artists ta
    INNER JOIN artists ar
        ON ta.artist_id = ar.artist_id
    WHERE ta.track_id = t.track_id
) AS artists
FROM tracks t
    INNER JOIN album al
        ON t.album_id = al.album_id
    LEFT JOIN genre g
        ON t.genre_id = g.genre_id
    LEFT JOIN track_feel tf
        ON t.track_id = tf.track_id
WHERE t.album_id = $1
    AND t.track_name = $2;

```

Query 2 — Find an exact album by title and release date

Type: Point query

Description:

This query retrieves detailed information about one particular album. A user might search for an album by its title and release date, and the application could then display the album's format, label, genre, and review scores.

The query begins with the album table and joins to genre to obtain the genre name. It also joins to album_scores to retrieve critic and user ratings from Metacritic and Album of the Year.

SQL:

```

SELECT
    al.album_id,
    al.album_name,
    al.album_format,
    al.release_date,    al.label,
    g.genre_name,
    s.metacritic_critic_score,
    s.metacritic_critic_count,
    s.metacritic_user_score,
    s.metacritic_user_count,
    s.aoty_critic_score,
    s.aoty_critic_count,
    s.aoty_user_score,

```

```
s.aoty_user_count
FROM album al
LEFT JOIN genre g
    ON al.genre_id = g.genre_id
LEFT JOIN album_scores s
    ON al.album_id = s.album_id
WHERE al.album_name = $1
AND al.release_date = $2;
```

Query 3 — Find popular tracks within a selected genre

Type: Range query

Description:

This query allows a user to choose a genre and then retrieve tracks whose popularity falls within a certain range.

For example, someone could ask for rock songs with popularity scores between 70 and 90. Instead of searching for one exact popularity value, the query searches across a range of values.

The results are sorted from most popular to least popular.

SQL:

```
SELECT
    t.track_id,
    t.track_name,
    g.genre_name,
    t.popularity,
    t.duration_ms,
    t.explicit
FROM tracks t
INNER JOIN genre g
    ON t.genre_id = g.genre_id
WHERE t.genre_id = $1
AND t.popularity BETWEEN $2 AND $3
ORDER BY t.popularity DESC;
```

Query 4 — Find highly rated albums released during a selected period

Type: Range query

Description:

This query finds albums from a particular genre that were released within a chosen date range and also received at least a certain critic score.

For example, a user could ask:

"Show me highly rated hip-hop albums released between 2015 and 2020 with a Metacritic critic score of at least 80."

The query therefore applies several filters at once: one exact genre, a range of release dates, and a minimum critic score. It then sorts the strongest-reviewed albums first.

SQL:

```
SELECT
    al.album_id,
    al.album_name,
    al.release_date,
    al.album_format,
    al.label,
    g.genre_name,
    s.metacritic_critic_score,
    s.metacritic_user_score
FROM album al
INNER JOIN genre g
    ON al.genre_id = g.genre_id
INNER JOIN album_scores s
    ON al.album_id = s.album_id
WHERE al.genre_id = $1
    AND al.release_date BETWEEN $2 AND $3
    AND s.metacritic_critic_score >= $4
ORDER BY
    s.metacritic_critic_score DESC,
    al.release_date DESC;
```

Query 5 — Compare the musical characteristics of different genres

Type: Scan / analytical query

Description:

This query is designed more for data analysis than for finding one particular record.

It looks at all tracks in each genre and calculates summary statistics such as:

- number of tracks,
- average popularity,
- average danceability,
- average energy,
- average tempo,
- percentage of tracks marked explicit.

The database therefore needs to examine many rows, join genre, tracks, and track_feel, and then group the results by genre.

SQL:

```
SELECT
  g.genre_id,
  g.genre_name,
  COUNT(*) AS track_count,
  ROUND(AVG(t.popularity)::numeric, 2) AS avg_popularity,
  ROUND(AVG(tf.danceability)::numeric, 3) AS avg_danceability,
  ROUND(AVG(tf.energy)::numeric, 3) AS avg_energy,
  ROUND(AVG(tf.tempo)::numeric, 2) AS avg_tempo,
  ROUND(
    100.0 * COUNT(*) FILTER (WHERE t.explicit IS TRUE)
    / NULLIF(COUNT(*), 0),
    2
  ) AS explicit_percentage
FROM genre g
INNER JOIN tracks t
  ON g.genre_id = t.genre_id
INNER JOIN track_feel tf
  ON t.track_id = tf.track_id
GROUP BY
  g.genre_id,
  g.genre_name
ORDER BY avg_popularity DESC;
```

Query 6 — Compare artists based on album review scores

Type: Scan / join-heavy analytical query

Description:

This query analyzes how well an artist's albums have been reviewed.

It calculates, for each artist:

- how many albums they have in the database,
- their average Metacritic critic score,
- their average Metacritic user score,
- their average Album of the Year critic score.

There is one complication. An artist may have many tracks on the same album. If we directly joined artists to tracks and then to album scores, the same album score could accidentally be counted several times.

To prevent that, the first part of the query creates a list containing each artist–album combination only once (CTE or view). SQL:

```
WITH artist_albums AS
(  SELECT DISTINCT
    ta.artist_id,
    t.album_id
    FROM track_artists ta
    INNER JOIN tracks t
        ON ta.track_id = t.track_id
    WHERE t.album_id IS NOT NULL
)
SELECT
    ar.artist_id,
    ar.artist_name,
    ar.artist_country,
    COUNT(aa.album_id) AS album_count,
    ROUND(
        AVG(s.metacritic_critic_score)::numeric,
        2
    ) AS avg_metacritic_critic_score,
    ROUND(
        AVG(s.metacritic_user_score)::numeric,
        2
    ) AS avg_metacritic_user_score,
    ROUND(
```

```

        AVG(s.aoty_critic_score)::numeric,
        2
    ) AS avg_aoty_critic_score
FROM artist_albums aa
INNER JOIN artists ar
    ON aa.artist_id = ar.artist_id
INNER JOIN album_scores s
    ON aa.album_id = s.album_id
GROUP BY
    ar.artist_id,
    ar.artist_name,
    ar.artist_country
HAVING COUNT(aa.album_id) >= 2
ORDER BY
    avg_metacritic_critic_score DESC NULLS LAST,
    album_count DESC;

```

For comparison, I also added two rather simple queries. One is exact lookup and the other is range.

Query 7 — Find a specific artist

Type: Point query

Description:

This query retrieves information about one specific artist using the artist's name and country. An application could use this when a user searches for an artist and wants to open that artist's profile.

SQL:

```

SELECT
    artist_id,
    artist_name,
    artist_country,
    artist_popularity
FROM artists
WHERE artist_name = $1
    AND artist_country = $2;

```

Query 8 — Find albums released during a selected period

Type: Range query

Description:

This query retrieves albums released between two dates. It could be used by someone exploring albums from a particular decade or period.

SQL:

```
SELECT
    album_id,
    album_name,
    album_format,
    release_date,
    label
FROM album
WHERE release_date BETWEEN $1 AND $2
ORDER BY release_date;
```